01 PROJECT TITLE

TalentMatch

AI-Assisted Resume-to-Job Matching and Recruitment Management System

02 STUDENT SRNS AND NAMES

SRN	NAME
PES1UG24AM229	Rohith Karning
PES1UG24AM237	Saanvi Maharana

Section D

GITHUB REPOSITORY
`github.com/rohithkarning-debug/TalentMatch`

DATABASE OPTION
SQL + Vector Database (handout §3, Option 2)

DATABASES
PostgreSQL 17 · pgvector 0.8.7

SUBMITTED
November 2026

CONTENTS

01	Project Title	1
02	Student SRNs and Names	1
03	Abstract	2
04	Technology Stack	3
05	SQL – Database Commands	4
06	NoSQL / Vector Database Commands	18
07	Next.js – Project Tree Structure	23
08	Sample Screenshots of Application Code and Demo	25
09	Evaluation Rubrics – Completion Status	34

03 Abstract

Recruiters often receive far more resumes than they can read, and keyword filters miss candidates who describe the right experience in different words. **TalentMatch** is a recruitment management web application that ranks candidates by the *meaning* of their resumes. Every resume and every job description is converted into a 384-dimensional sentence embedding with the all-MiniLM-L6-v2 model, run locally on the server, and stored in PostgreSQL using the **pgvector** extension. Ranking candidates for a job is a cosine-similarity query (`<=>`) executed inside the database, joined with the relational data it depends on.

Around the matching sits a complete hiring workflow for three roles. **Candidates** register, manage their profile and skills, upload PDF or text resumes, search and filter jobs, see their skill gap for a job, apply, and track every status change. **Recruiters** post jobs with required skills, review applicants ranked by similarity, move applications through applied → shortlisted → interviewed → offered (or rejected), schedule interviews and make offers. **Admins** manage companies and recruiter accounts and view hiring analytics.

The database enforces the business rules instead of trusting the interface: 16 tables with 22 foreign keys, 57 CHECK and 8 UNIQUE constraints, 46 indexes (2 of them HNSW vector indexes), 19 triggers, 6 stored functions, 3 views and explicit transactions. Invalid status changes, duplicate applications, non-unit vectors and unsafe deletes are refused by PostgreSQL itself. On the seeded data, the Machine Learning Engineer job ranks Aditi Sharma first (similarity 0.7819, 3 of 3 required skills) and a frontend developer last (0.2438), while the Frontend Developer job reverses that order. The final test run passed 85 of 85 unit tests and 147 of 147 integration tests, plus the SQL constraint and advanced-feature test scripts.

3	16	24	36	19
user roles	tables	migrations	API route files	app pages

Keywords: PostgreSQL, pgvector, semantic search, sentence embeddings, cosine similarity, HNSW, triggers, transactions, Next.js

04 Technology Stack

LAYER	TECHNOLOGY	USED FOR
Frontend	Next.js 16.4.0 (App Router), React 19.3.0, Tailwind CSS 4	19 pages for the three roles: forms, tables, search, filters, dashboards
Backend / API	Next.js Route Handlers	36 `route.js` files under `src/app/api`; every protected handler checks the session and role on the server
Relational database	PostgreSQL 17 (Docker image `pgvector/pgvector:pg17`)	Users, companies, jobs, applications, interviews, offers, notifications
Vector database	pgvector 0.8.7 (same PostgreSQL database)	`vector(384)` columns, HNSW indexes, cosine distance `<=>`
Database driver	`pg` (node-postgres) 8.23	Connection pool; raw parameterised SQL, no ORM
Embedding model	`@huggingface/transformers` 4.3 running `all-MiniLM-L6-v2`	Resume and job embeddings, computed locally (no external API)
Resume parsing	`pdfjs-dist` 6.4	Text extraction from uploaded PDFs
Validation	`zod` 4.6	Request validation in the API layer (the database validates again)
Authentication	`bcryptjs` 3.0, `jose` 6.2	bcrypt password hashes (cost 12); HS256 JWT in an httpOnly cookie
Testing	`node:test` (built in), SQL test scripts	85 unit and 147 integration tests; constraint and advanced-feature SQL tests
Tooling	Docker Compose, Git and GitHub	Local database container; version control

Versions are taken from `package.json`, `docker-compose.yml` and the test environment recorded in `docs/test-cases.md`.

05 SQL – Database Commands

Every command in this section is copied from the repository file named in its panel header, with the real line numbers. The schema is built by 24 ordered migrations in `db/migrations/` (`npm run db:migrate`), each run in its own transaction. Outputs were produced by running the same commands with `psql` on a database built from those migrations and `db/seed/seed.sql`. Object counts below come from the PostgreSQL system catalogs.

5.1 Schema at a glance

16	22	57	8	46	19	6	3
tables	foreign keys	CHECK	UNIQUE	indexes	triggers	functions	views

TABLE	HOLDS	KEYS AND NOTABLE CONSTRAINTS
`users`	Login account for every role	UNIQUE email; CHECK lower-case email, bcrypt hash format, role
`companies`	Hiring companies	UNIQUE name; CHECK website starts with http(s)
`recruiters`	Recruiter profile (1 : 1 user)	FK (user_id, role) → users; FK company RESTRICT
`candidates`	Candidate profile (1 : 1 user)	FK (user_id, role) → users; CHECK experience 0–60
`skills`	Master list of skills	CHECK category
`candidate_skills`	Candidate ↔ skill (M : N)	PK (candidate_id, skill_id); proficiency 1–5
`jobs`	Job postings	12 CHECKs: salary range, status, work mode, closing date…
`job_skills`	Job ↔ skill (M : N)	PK (job_id, skill_id); required flag, minimum proficiency
`resumes`	Uploaded resumes and extracted text	Partial unique index: one primary resume per candidate
`applications`	A candidate applying to a job	UNIQUE (job_id, candidate_id); status CHECK; match_score −1…1
`application_status_history`	Audit trail of status changes	Written by a trigger; CHECK old ≠ new status
`interviews`	Interview rounds	UNIQUE (application_id, round_no); 7 CHECKs
`offers`	Job offers	UNIQUE application_id (one offer per application)
`notifications`	In-app messages	FK user, FK application; CHECK type
`resume_embeddings`	Resume vector	PK = FK → resumes CASCADE; unit-length CHECK; HNSW
`job_embeddings`	Job vector	PK = FK → jobs CASCADE; unit-length CHECK; HNSW

5.2 DDL — CREATE, constraints, ALTER, DROP, indexes

`CREATE TABLE` users — primary key, UNIQUE and CHECK constraints

Every account. The CHECKs reject an upper-case or malformed email, a password that is not a bcrypt hash, and an unknown role. `UNIQUE (user_id, role)` is the target of the composite foreign keys from `recruiters` and `candidates`.

db/migrations/002_users_companies_profiles.sql — lines 7–28

```sql
CREATE TABLE IF NOT EXISTS users (
    user_id        BIGINT GENERATED ALWAYS AS IDENTITY,
    email          VARCHAR(255) NOT NULL,
    password_hash  VARCHAR(100) NOT NULL,
    full_name      VARCHAR(120) NOT NULL,
    role           VARCHAR(20)  NOT NULL,
    is_active      BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT users_pkey PRIMARY KEY (user_id),
    CONSTRAINT users_email_key UNIQUE (email),
    -- Emails are stored lower-case so 'A@x.com' and 'a@x.com' cannot both exist.
    CONSTRAINT users_email_format_check
        CHECK (email = lower(email) AND email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
    -- Only bcrypt hashes are accepted, never plaintext passwords.
    CONSTRAINT users_password_hash_bcrypt_check
        CHECK (password_hash ~ '^\$2[aby]\$[0-9]{2}\$.{53}$'),
    CONSTRAINT users_full_name_not_blank_check CHECK (btrim(full_name) <> ''),
    CONSTRAINT users_role_check CHECK (role IN ('admin', 'recruiter', 'candidate')),
    -- Target for the (user_id, role) foreign keys in recruiters/candidates.
    CONSTRAINT users_user_id_role_key UNIQUE (user_id, role)
);
```

`CREATE TABLE` applications — foreign keys with different delete rules

One row per application. A job with applications cannot be deleted (RESTRICT, close it instead); deleting a candidate removes their applications (CASCADE); deleting a resume keeps the application (SET NULL).

db/migrations/005_resumes_applications.sql — lines 29–59

```sql
CREATE TABLE IF NOT EXISTS applications (
    application_id  BIGINT GENERATED ALWAYS AS IDENTITY,
    job_id          BIGINT       NOT NULL,
    candidate_id    BIGINT       NOT NULL,
    resume_id       BIGINT,
    cover_letter    TEXT,
    status          VARCHAR(15)  NOT NULL DEFAULT 'applied',
    -- Cosine similarity between resume and job embeddings (1 - cosine distance).
    -- NULL until computed from real embeddings; never filled with fake values.
    match_score     NUMERIC(5,4),
    applied_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT applications_pkey PRIMARY KEY (application_id),
    -- A candidate can apply to a given job only once.
    CONSTRAINT applications_job_id_candidate_id_key UNIQUE (job_id, candidate_id),
    CONSTRAINT applications_status_check CHECK (status IN
        ('applied', 'shortlisted', 'interviewed', 'offered', 'rejected')),
    CONSTRAINT applications_match_score_check CHECK (
        match_score IS NULL OR match_score BETWEEN -1 AND 1),
    CONSTRAINT applications_updated_after_applied_check CHECK (updated_at >= applied_at),
    -- Jobs with applications cannot be deleted; close them instead.
    CONSTRAINT applications_job_fkey FOREIGN KEY (job_id)
        REFERENCES jobs (job_id) ON DELETE RESTRICT,
    -- Deleting a candidate account removes their applications (privacy).
    CONSTRAINT applications_candidate_fkey FOREIGN KEY (candidate_id)
        REFERENCES candidates (user_id) ON DELETE CASCADE,
    -- Deleting a resume file keeps the application record.
    CONSTRAINT applications_resume_fkey FOREIGN KEY (resume_id)
        REFERENCES resumes (resume_id) ON DELETE SET NULL
);
```

`CREATE TABLE` recruiters — composite foreign key

`(user_id, role) → users (user_id, role)` guarantees that only a user whose role is recruiter can have a recruiter profile.

db/migrations/002_users_companies_profiles.sql — lines 55–68

```
55      CONSTRAINT recruiters_pkey PRIMARY KEY (user_id),
56      CONSTRAINT recruiters_role_check CHECK (role = 'recruiter'),
57      CONSTRAINT recruiters_phone_check CHECK (phone IS NULL OR phone ~ '^\+?[0-9][0-9 -]{6,18}$'),
58      -- Deleting the user account deletes the recruiter profile.
59      -- Changing the user's role is blocked while this profile exists (NO ACTION).
60      CONSTRAINT recruiters_user_fkey FOREIGN KEY (user_id, role)
61          REFERENCES users (user_id, role) ON DELETE CASCADE,
62      -- A company that still employs recruiters cannot be deleted.
63      CONSTRAINT recruiters_company_fkey FOREIGN KEY (company_id)
64          REFERENCES companies (company_id) ON DELETE RESTRICT,
65      -- Target for jobs (recruiter_id, company_id) FK: a recruiter can only own
66      -- jobs of their own company.
67      CONSTRAINT recruiters_user_id_company_id_key UNIQUE (user_id, company_id)
68  );
```

`CONSTRAINTS IN ACTION` CHECK and UNIQUE violations

Two invalid writes against the seeded data. PostgreSQL refuses both, whatever the application does.

invalid role

```
INSERT INTO users (email, password_hash, full_name, role)
VALUES ('test.user@mail.example',
        '$2b$10$qYCMCdmT.qdfC2q8kd4WZuYcxb3.8emfuj2VIVZnakezYQb7rgtyC',
        'Test User', 'superuser');
```

OUTPUT — psql, seeded TalentMatch database

```
ERROR:  new row for relation "users" violates check constraint "users_role_check"
```

second application to the same job

```
INSERT INTO applications (job_id, candidate_id)
SELECT job_id, candidate_id FROM applications WHERE application_id = 2;
```

OUTPUT — psql, seeded TalentMatch database

```
ERROR:  duplicate key value violates unique constraint "applications_job_id_candidate_id_key"
DETAIL:  Key (job_id, candidate_id)=(2, 13) already exists.
```

`ALTER TABLE` schema evolution after the first release

New columns with defaults and inline CHECKs, a column type change, and named constraints added after the fact.

db/migrations/009_alter_tables.sql — lines 5–29

```sql
-- 1. New column with a default and an inline CHECK constraint.
ALTER TABLE jobs
    ADD COLUMN IF NOT EXISTS employment_type VARCHAR(20) NOT NULL DEFAULT 'full_time'
        CONSTRAINT jobs_employment_type_check
        CHECK (employment_type IN ('full_time', 'part_time', 'contract', 'internship'));

-- 2. Minimum experience required by a job, validated against a sensible range.
ALTER TABLE jobs
    ADD COLUMN IF NOT EXISTS min_experience_years NUMERIC(4,1) NOT NULL DEFAULT 0;
ALTER TABLE jobs DROP CONSTRAINT IF EXISTS jobs_min_experience_check;
ALTER TABLE jobs
    ADD CONSTRAINT jobs_min_experience_check CHECK (min_experience_years BETWEEN 0 AND 40);

-- 3. Track the last successful login (filled in by the auth module later).
ALTER TABLE users ADD COLUMN IF NOT EXISTS last_login_at TIMESTAMPTZ;

-- 4. Change an existing column: allow longer company industry labels.
ALTER TABLE companies ALTER COLUMN industry TYPE VARCHAR(120);

-- 5. Add a named constraint after the fact: a closing date cannot be before
--    the posting date.
ALTER TABLE jobs DROP CONSTRAINT IF EXISTS jobs_closes_after_posted_check;
ALTER TABLE jobs
    ADD CONSTRAINT jobs_closes_after_posted_check
    CHECK (closes_at IS NULL OR closes_at >= (posted_at AT TIME ZONE 'UTC')::date);
```

`CREATE INDEX` B-tree, composite and partial indexes

Indexes on foreign-key and filter columns. The two partial indexes only cover rows hot queries touch: open jobs, newest first, and unread notifications. Foreign-key columns are indexed in lines 8–19 of the same file; the two HNSW vector indexes are in Section 6.

db/migrations/008_indexes.sql — lines 21–35

```sql
-- Frequently queried columns
CREATE INDEX IF NOT EXISTS users_role_idx                 ON users (role);
CREATE INDEX IF NOT EXISTS applications_job_status_idx    ON applications (job_id, status);
CREATE INDEX IF NOT EXISTS interviews_scheduled_at_idx    ON interviews (scheduled_at);
CREATE INDEX IF NOT EXISTS notifications_user_created_idx ON notifications (user_id, created_at DESC);

-- Partial indexes: only index the rows that hot queries actually touch.
-- Candidates browse OPEN jobs, newest first; closed/draft jobs are excluded,
-- keeping the index small.
CREATE INDEX IF NOT EXISTS jobs_open_posted_at_idx
    ON jobs (posted_at DESC) WHERE status = 'open';

-- The notification bell only counts UNREAD notifications.
CREATE INDEX IF NOT EXISTS notifications_unread_idx
    ON notifications (user_id) WHERE NOT is_read;
```

`DROP` reset script

`npm run db:reset` drops the views first, then the tables children-first, then every function and the migration log.

db/reset.sql — lines 11–30, 40–42, 53–54

```
11  DROP VIEW IF EXISTS company_hiring_summary;
12  DROP VIEW IF EXISTS candidate_application_view;
13  DROP VIEW IF EXISTS recruiter_pipeline_view;
14
15  DROP TABLE IF EXISTS job_embeddings             CASCADE;
16  DROP TABLE IF EXISTS resume_embeddings          CASCADE;
17  DROP TABLE IF EXISTS notifications              CASCADE;
18  DROP TABLE IF EXISTS offers                     CASCADE;
19  DROP TABLE IF EXISTS interviews                 CASCADE;
20  DROP TABLE IF EXISTS application_status_history CASCADE;
21  DROP TABLE IF EXISTS applications               CASCADE;
22  DROP TABLE IF EXISTS resumes                    CASCADE;
23  DROP TABLE IF EXISTS job_skills                 CASCADE;
24  DROP TABLE IF EXISTS jobs                       CASCADE;
25  DROP TABLE IF EXISTS candidate_skills           CASCADE;
26  DROP TABLE IF EXISTS skills                     CASCADE;
27  DROP TABLE IF EXISTS candidates                 CASCADE;
28  DROP TABLE IF EXISTS recruiters                 CASCADE;
29  DROP TABLE IF EXISTS companies                  CASCADE;
30  DROP TABLE IF EXISTS users                      CASCADE;
⋮
40  DROP FUNCTION IF EXISTS compute_match_score(BIGINT, BIGINT);
41  DROP FUNCTION IF EXISTS rank_candidates_for_job(BIGINT, INTEGER, BOOLEAN);
42  DROP FUNCTION IF EXISTS update_application_status(BIGINT, VARCHAR, BIGINT, TEXT);
⋮
53  DROP TABLE  IF EXISTS tm_meta.schema_migrations;
54  DROP SCHEMA IF EXISTS tm_meta;
```

5.3 DML — INSERT, UPDATE, DELETE, SELECT

`INSERT` seed data, idempotent

The seed runs as one transaction and can be re-run safely because of `ON CONFLICT … DO NOTHING`. Company names, domains and people in the seed are fictional.

db/seed/seed.sql — lines 70–80

```
70  INSERT INTO companies (name, industry, location, website, description) VALUES
71      ('Nimbus Analytics Pvt Ltd', 'Data & AI Consulting', 'Bengaluru, Karnataka',
72       'https://nimbusanalytics.example',
73       'Builds analytics platforms and machine-learning solutions for retail and logistics clients.'),
74      ('Kestrel Software Labs', 'SaaS Product Development', 'Pune, Maharashtra',
75       'https://kestrellabs.example',
76       'Product company building collaboration and scheduling software for small businesses.'),
77      ('Orbitline Cloud Systems', 'Cloud Infrastructure', 'Hyderabad, Telangana',
78       'https://orbitline.example',
79       'Managed cloud and platform-engineering services for fintech and healthcare customers.')
80  ON CONFLICT (name) DO NOTHING;
```

`UPDATE` from a Route Handler, parameterised

Marks the logged-in user's unread notifications as read. `$1` is bound by the driver, never concatenated into the SQL.

src/app/api/notifications/read-all/route.js — lines 8–13

```
8   export const POST = withAuth({}, async (request, { user }) => {
9     assertSameOrigin(request);
10    const { rowCount } = await query(
11      "UPDATE notifications SET is_read = TRUE WHERE user_id = $1 AND NOT is_read", [user.id]);
12    return NextResponse.json({ updated: rowCount }, { headers: { "Cache-Control": "no-store" } });
13  });
```

`DELETE` inside a transaction, with ownership in the WHERE clause

Deleting a resume: the candidate id comes from the session, so another user's resume is simply not found. If the primary resume is deleted, the newest remaining one is promoted in the same transaction.

src/app/api/candidate/resumes/[resumeId]/route.js — lines 14–31

```
14    const client = await getPool().connect();
15    let storedPath;
16    try {
17      await client.query("BEGIN");
18      await lockCandidateResumes(client, user.id);
19      const found = await client.query("SELECT storage_path, is_primary FROM resumes WHERE resume_id = $1 AND
    candidate_id = $2", [parsed.data, user.id]);
20      if (!found.rowCount) throw notFound("Resume not found");
21      storedPath = found.rows[0].storage_path;
22      await client.query("DELETE FROM resumes WHERE resume_id = $1 AND candidate_id = $2", [parsed.data,
    user.id]);
23      // Deleting the primary resume promotes the newest remaining one.
24      if (found.rows[0].is_primary) {
25        await client.query("UPDATE resumes SET is_primary = TRUE WHERE resume_id = (SELECT resume_id FROM resumes
    WHERE candidate_id = $1 ORDER BY uploaded_at DESC, resume_id DESC LIMIT 1)", [user.id]);
26      }
27      await client.query("COMMIT");
28    } catch (err) {
29      await client.query("ROLLBACK");
30      throw err;
31    } finally { client.release(); }
```

`DELETE` refused by ON DELETE RESTRICT

A company that still has recruiters and jobs cannot be deleted (query Q41).

db/queries/important_queries.sql — lines 780–784

```
780  DO $$ BEGIN
781      DELETE FROM companies WHERE name = 'Nimbus Analytics Pvt Ltd';
782  EXCEPTION WHEN foreign_key_violation THEN
783      RAISE NOTICE 'refused: %', SQLERRM;
784  END $$;
```

OUTPUT — psql, seeded TalentMatch database

```
NOTICE:  refused: update or delete on table "companies" violates foreign key constraint "recruiters_company_fkey"
on table "recruiters"
```

`SELECT · JOIN` open jobs with company and recruiter (Q1)

INNER JOIN to companies, LEFT JOIN to users because a job's recruiter can be NULL. Served by the partial index on open jobs.

db/queries/important_queries.sql — lines 19–25

```
19  SELECT j.job_id, j.title, c.name AS company, u.full_name AS recruiter,
20         j.location, j.work_mode, j.posted_at::date AS posted
21  FROM jobs j
22  JOIN companies c       ON c.company_id = j.company_id
23  LEFT JOIN users u      ON u.user_id = j.recruiter_id   -- recruiter may be NULL
24  WHERE j.status = 'open'
25  ORDER BY j.posted_at DESC;
```

GROUP BY · WINDOW

applications per status (Q2)

COUNT per group, and each group's share of the total with a window function over the aggregate.

db/queries/important_queries.sql — lines 31–36

```
31  SELECT status,
32         count(*)                                                AS applications,
33         round(100.0 * count(*) / sum(count(*)) OVER (), 1)      AS pct_of_total
34  FROM applications
35  GROUP BY status
36  ORDER BY array_position(ARRAY['applied','shortlisted','interviewed','offered','rejected'], status::text);
```

OUTPUT — psql, seeded TalentMatch database

```
   status    | applications | pct_of_total
-------------+--------------+--------------
 applied     |            6 |         33.3
 shortlisted |            3 |         16.7
 interviewed |            3 |         16.7
 offered     |            2 |         11.1
 rejected    |            4 |         22.2
(5 rows)
```

GROUP BY · HAVING

jobs with at least 3 applicants (Q3)

HAVING filters on the aggregate after grouping.

db/queries/important_queries.sql — lines 42–48

```
42  SELECT c.name AS company, j.title, count(a.application_id) AS applicants
43  FROM jobs j
44  JOIN companies c     ON c.company_id = j.company_id
45  JOIN applications a  ON a.job_id = j.job_id
46  GROUP BY c.name, j.job_id, j.title
47  HAVING count(a.application_id) >= 3
48  ORDER BY applicants DESC, j.title;
```

OUTPUT — psql, seeded TalentMatch database

```
         company          |                title                | applicants
--------------------------+-------------------------------------+------------
 Nimbus Analytics Pvt Ltd | Machine Learning Engineer           |          4
 Kestrel Software Labs    | Backend Engineer (Node.js)          |          3
 Nimbus Analytics Pvt Ltd | Data Analyst                        |          3
 Kestrel Software Labs    | Frontend Developer (React / Next.js) |          3
(4 rows)
```

AGGREGATES **salary statistics per company (Q4)**

MIN, MAX, AVG and COUNT with FILTER. Salaries are INR per year.

db/queries/important_queries.sql — lines 54–63

```
54  SELECT c.name AS company,
55         count(*)                                    AS jobs,
56         count(*) FILTER (WHERE j.status = 'open')   AS open_jobs,
57         min(j.salary_min)                           AS lowest_min,
58         max(j.salary_max)                           AS highest_max,
59         round(avg((j.salary_min + j.salary_max) / 2))  AS avg_midpoint
60  FROM jobs j
61  JOIN companies c ON c.company_id = j.company_id
62  GROUP BY c.name
63  ORDER BY avg_midpoint DESC;
```

OUTPUT — psql, seeded TalentMatch database

```
         company          | jobs | open_jobs | lowest_min | highest_max | avg_midpoint
--------------------------+------+-----------+------------+-------------+--------------
 Orbitline Cloud Systems  |    2 |         1 | 1400000.00 |  3200000.00 |      2200000
 Kestrel Software Labs    |    3 |         2 | 1000000.00 |  2000000.00 |      1450000
 Nimbus Analytics Pvt Ltd |    3 |         3 |  300000.00 |  2800000.00 |      1220000
(3 rows)
```

SUBQUERY **candidates with above-average experience (Q9)**

Scalar subqueries in both the WHERE clause and the SELECT list.

db/queries/important_queries.sql — lines 123–128

```
123  SELECT u.full_name, cd.years_experience,
124         (SELECT round(avg(years_experience), 2) FROM candidates) AS platform_avg
125  FROM candidates cd
126  JOIN users u ON u.user_id = cd.user_id
127  WHERE cd.years_experience > (SELECT avg(years_experience) FROM candidates)
128  ORDER BY cd.years_experience DESC;
```

OUTPUT — psql, seeded TalentMatch database

```
      full_name      | years_experience | platform_avg
---------------------+------------------+--------------
 Karthik Subramanian |              6.0 |         3.08
 Rohan Verma         |              5.0 |         3.08
 Vikram Chauhan      |              4.5 |         3.08
 Neha Bhatt          |              4.0 |         3.08
(4 rows)
```

SUBQUERY IN HAVING **companies with above-average applications (Q38)**

A GROUP BY whose HAVING compares each group with a computed average.

db/queries/important_queries.sql — lines 736–742

```
736  SELECT c.name, count(a.application_id) AS applications
737    FROM companies c
738    LEFT JOIN jobs j         ON j.company_id = c.company_id
739    LEFT JOIN applications a ON a.job_id = j.job_id
740   GROUP BY c.company_id, c.name
741  HAVING count(a.application_id) > (SELECT count(*)::numeric / (SELECT count(*) FROM companies) FROM applications)
742   ORDER BY applications DESC;
```

OUTPUT — psql, seeded TalentMatch database

```
           name           | applications
--------------------------+--------------
 Nimbus Analytics Pvt Ltd |            8
(1 row)
```

The full set of 41 documented queries (JOINs, NOT EXISTS, correlated subqueries, EXCEPT, LAG, views, functions, transactions) is in `db/queries/important_queries.sql`; the whole file runs with exit code 0.

5.4 Advanced features — views, stored functions, triggers

VIEW company_hiring_summary

One row per company with hiring activity: jobs, applications, offers, rejection rate. Used by the admin dashboard. `count(DISTINCT j.job_id)` because each job repeats once per application in the join. The other two views are `recruiter_pipeline_view` and `candidate_application_view`.

db/migrations/013_views.sql — lines 118–141

```
118  CREATE OR REPLACE VIEW company_hiring_summary AS
119  SELECT
120      c.company_id,
121      c.name                                                  AS company_name,
122      c.industry,
123      count(DISTINCT j.job_id)                                AS total_jobs,
124      count(DISTINCT j.job_id) FILTER (WHERE j.status = 'open')    AS open_jobs,
125      count(a.application_id)                                 AS total_applications,
126      count(DISTINCT a.candidate_id)                          AS unique_candidates,
127      round(count(a.application_id)::numeric
128            / nullif(count(DISTINCT j.job_id), 0), 2)         AS avg_applications_per_job,
129      count(a.application_id) FILTER (WHERE a.status = 'offered')  AS currently_offered,
130      count(a.application_id) FILTER (WHERE a.status = 'rejected') AS rejected,
131      round(100.0 * count(a.application_id) FILTER (WHERE a.status = 'rejected')
132            / nullif(count(a.application_id), 0), 1)          AS rejection_rate_pct,
133      count(o.offer_id)                                       AS offers_made,
134      count(o.offer_id) FILTER (WHERE o.status = 'accepted')  AS offers_accepted,
135      round(avg(a.match_score), 4)                            AS avg_match_score
136  FROM companies c
137  JOIN jobs j               ON j.company_id = c.company_id
138  LEFT JOIN applications a  ON a.job_id = j.job_id
139  LEFT JOIN offers o        ON o.application_id = a.application_id
140  GROUP BY c.company_id, c.name, c.industry
141  HAVING count(a.application_id) > 0;
```

querying the view

```
SELECT company_name, total_jobs AS jobs, total_applications AS applications,
       unique_candidates AS candidates, offers_made AS offers,
       offers_accepted AS accepted, rejection_rate_pct AS rejected_pct
  FROM company_hiring_summary ORDER BY company_name;
```

OUTPUT — psql, seeded TalentMatch database

```
       company_name       | jobs | applications | candidates | offers | accepted | rejected_pct
--------------------------+------+--------------+------------+--------+----------+--------------
 Kestrel Software Labs    |    3 |            6 |          5 |      1 |        0 |         33.3
 Nimbus Analytics Pvt Ltd |    3 |            8 |          6 |      1 |        0 |         12.5
 Orbitline Cloud Systems  |    2 |            4 |          2 |      1 |        1 |         25.0
(3 rows)
```

FUNCTION update_application_status() — the only way the app changes a status

PL/pgSQL. Validates the new status, locks the row with FOR UPDATE, checks that the caller may change it (candidates never, recruiters only for their own company), passes who and why to the history trigger, updates, and notifies the candidate.

db/migrations/012_functions.sql — lines 128–136, 151–159, 165–177, 184–205

```
128  CREATE OR REPLACE FUNCTION update_application_status(
129      p_application_id BIGINT,
130      p_new_status     VARCHAR,
131      p_changed_by     BIGINT,
132      p_note           TEXT DEFAULT NULL
133  )
```

```
RETURNS applications
LANGUAGE plpgsql
AS $$
⋮
    -- 2. Load and lock the application
    SELECT * INTO v_app
      FROM applications
     WHERE application_id = p_application_id
       FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Application % does not exist', p_application_id
            USING ERRCODE = 'no_data_found';
    END IF;
⋮
    -- 3. Authorisation (defence in depth; the API checks too)
    SELECT role INTO v_actor_role FROM users WHERE user_id = p_changed_by AND is_active;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'User % does not exist or is inactive', p_changed_by
            USING ERRCODE = 'insufficient_privilege';
    END IF;
    IF v_actor_role = 'candidate'
       OR (v_actor_role = 'recruiter' AND NOT EXISTS (
              SELECT 1 FROM recruiters r
               WHERE r.user_id = p_changed_by AND r.company_id = v_company_id)) THEN
        RAISE EXCEPTION 'User % is not allowed to update application %', p_changed_by, p_application_id
            USING ERRCODE = 'insufficient_privilege';
    END IF;
⋮
    -- 4. Pass who/why to the history trigger, then update.
    PERFORM set_config('talentmatch.actor_id', p_changed_by::text, true);
    PERFORM set_config('talentmatch.status_note', coalesce(p_note, ''), true);

    UPDATE applications
       SET status = p_new_status
     WHERE application_id = p_application_id
    RETURNING * INTO v_app;

    PERFORM set_config('talentmatch.actor_id', '', true);
    PERFORM set_config('talentmatch.status_note', '', true);

    -- 5. Notify the candidate
    INSERT INTO notifications (user_id, type, title, message, related_application_id)
    VALUES (v_app.candidate_id, 'status_changed',
            'Application update: ' || v_job_title,
            format('Your application for %s moved from %s to %s.', v_job_title, v_old_status, p_new_status),
            p_application_id);

    RETURN v_app;
END;
$$;
```

calling it inside a rolled-back transaction

```
BEGIN;
SELECT application_id, status
  FROM update_application_status(2, 'shortlisted',
         (SELECT user_id FROM users WHERE email = 'meera.iyer@nimbusanalytics.example'),
         'Strong SQL portfolio');
SELECT old_status, new_status, note
  FROM application_status_history WHERE application_id = 2 ORDER BY changed_at;
ROLLBACK;
```

OUTPUT psql, seeded TalentMatch database

```
BEGIN
 application_id |   status
----------------+-------------
              2 | shortlisted
(1 row)

 old_status | new_status  |         note
------------+-------------+----------------------
            | applied     | Application submitted
 applied    | shortlisted | Strong SQL portfolio
(2 rows)

ROLLBACK
```

TRIGGER status-transition guard (BEFORE UPDATE)

The allowed workflow is a function; the trigger calls it before the row is written, so an invalid change never reaches the table, whether it comes from the app or from a manual UPDATE.

db/migrations/011_application_triggers.sql lines 40–53, 111–130

```
40  CREATE OR REPLACE FUNCTION is_valid_status_transition(p_old VARCHAR, p_new VARCHAR)
41  RETURNS boolean
42  LANGUAGE sql
43  IMMUTABLE
44  AS $$
45      SELECT CASE
46          WHEN p_old IS NOT DISTINCT FROM p_new THEN TRUE           -- no change
47          WHEN p_old = 'applied'     THEN p_new IN ('shortlisted', 'rejected')
48          WHEN p_old = 'shortlisted' THEN p_new IN ('interviewed', 'rejected')
49          WHEN p_old = 'interviewed' THEN p_new IN ('offered', 'rejected')
50          WHEN p_old = 'offered'     THEN p_new = 'rejected'
51          ELSE FALSE                                               -- 'rejected' is final
52      END;
53  $$;
 ⋮
111 CREATE OR REPLACE FUNCTION guard_application_status_transition()
112 RETURNS trigger
113 LANGUAGE plpgsql
114 AS $$
115 BEGIN
116     IF NOT is_valid_status_transition(OLD.status, NEW.status) THEN
117         RAISE EXCEPTION 'Invalid application status transition: % -> % (application %)',
118                         OLD.status, NEW.status, OLD.application_id
119             USING ERRCODE = 'check_violation',
120                   HINT = 'Allowed: applied->shortlisted|rejected, shortlisted->interviewed|rejected, '
121                          'interviewed->offered|rejected, offered->rejected. rejected is final.';
122     END IF;
123     RETURN NEW;
124 END;
125 $$;
126
127 DROP TRIGGER IF EXISTS trg_applications_status_guard ON applications;
128 CREATE TRIGGER trg_applications_status_guard
129     BEFORE UPDATE OF status ON applications
130     FOR EACH ROW EXECUTE FUNCTION guard_application_status_transition();
```

skipping two stages

```
UPDATE applications SET status = 'offered' WHERE application_id = 2;  -- currently 'applied'
```

OUTPUT psql, seeded TalentMatch database

```
ERROR:  Invalid application status transition: applied -> offered (application 2)
HINT:  Allowed: applied->shortlisted|rejected, shortlisted->interviewed|rejected, interviewed->offered|rejected,
offered->rejected. rejected is final.
CONTEXT:  PL/pgSQL function guard_application_status_transition() line 4 at RAISE
```

TRIGGER applicant_count kept exact (AFTER INSERT / DELETE)

`jobs.applicant_count` is never written by the application. This row trigger adds or subtracts 1, including for cascaded deletes.

db/migrations/011_application_triggers.sql — lines 70–85, 97–100

```
70  CREATE OR REPLACE FUNCTION maintain_applicant_count()
71  RETURNS trigger
72  LANGUAGE plpgsql
73  AS $$
74  BEGIN
75      IF TG_OP = 'INSERT' THEN
76          UPDATE jobs SET applicant_count = applicant_count + 1 WHERE job_id = NEW.job_id;
77      ELSIF TG_OP = 'DELETE' THEN
78          UPDATE jobs SET applicant_count = applicant_count - 1 WHERE job_id = OLD.job_id;
79      ELSIF TG_OP = 'UPDATE' AND NEW.job_id IS DISTINCT FROM OLD.job_id THEN
80          UPDATE jobs SET applicant_count = applicant_count - 1 WHERE job_id = OLD.job_id;
81          UPDATE jobs SET applicant_count = applicant_count + 1 WHERE job_id = NEW.job_id;
82      END IF;
83      RETURN NULL;  -- return value of an AFTER trigger is ignored
84  END;
85  $$;
 ⋮
97  DROP TRIGGER IF EXISTS trg_applications_applicant_count ON applications;
98  CREATE TRIGGER trg_applications_applicant_count
99      AFTER INSERT OR DELETE OR UPDATE OF job_id ON applications
100     FOR EACH ROW EXECUTE FUNCTION maintain_applicant_count();
```

CONSTRAINT TRIGGER deferred check: interviewed needs an interview

DEFERRABLE INITIALLY DEFERRED runs the check at COMMIT, after every statement of the transaction has run. Q29 forces it early with SET CONSTRAINTS … IMMEDIATE.

db/migrations/018_interview_offer_rules.sql — lines 214–220

```
214 DROP TRIGGER IF EXISTS trg_applications_stage_records ON applications;
215 CREATE CONSTRAINT TRIGGER trg_applications_stage_records
216     AFTER UPDATE OF status ON applications
217     DEFERRABLE INITIALLY DEFERRED
218     FOR EACH ROW
219     WHEN (NEW.status IN ('interviewed', 'offered') AND NEW.status IS DISTINCT FROM OLD.status)
220     EXECUTE FUNCTION check_application_stage_records();
```

db/queries/important_queries.sql — lines 563–576

```
563 BEGIN;
564 DO $$
565 BEGIN
566     PERFORM update_application_status(
567         (SELECT a.application_id FROM applications a JOIN users u ON u.user_id = a.candidate_id
568           JOIN jobs j ON j.job_id = a.job_id
569          WHERE u.email = 'ishita.banerjee@mail.example' AND j.title = 'Data Analyst'),
570         'interviewed', (SELECT user_id FROM users WHERE email = 'meera.iyer@nimbusanalytics.example'), 'no
    interview');
571     RAISE NOTICE 'UPDATE shortlisted -> interviewed accepted for now (check deferred to COMMIT)';
572     SET CONSTRAINTS trg_applications_stage_records IMMEDIATE;
573 EXCEPTION WHEN check_violation THEN
574     RAISE NOTICE 'deferred check: %', SQLERRM;
575 END $$;
576 ROLLBACK;
```

OUTPUT — psql, seeded TalentMatch database

```
NOTICE:  UPDATE shortlisted -> interviewed accepted for now (check deferred to COMMIT)
NOTICE:  deferred check: Application 3 cannot be interviewed without an interview; schedule one instead
```

5.5 Transactions

Server code opens explicit `BEGIN … COMMIT / ROLLBACK` transactions in 10 files (13 `BEGIN` calls): registration, profile update, resume upload and delete, job create/edit/delete, recruiter accounts, applying, interviews and offers. The applying transaction is shown in Section 8. The demo script below shows the database side: when one statement fails, nothing from the transaction survives.

`BEGIN · ROLLBACK` a failing transaction leaves no trace (transaction_demo.sql, part B)

Register a candidate and apply, then apply again: the UNIQUE (job_id, candidate_id) constraint fails, PostgreSQL aborts the transaction, and ROLLBACK removes the user, the profile, the application, the trigger-written history row and the applicant_count change.

db/demos/transaction_demo.sql — lines 56–83

```
56  BEGIN;
57    INSERT INTO users (email, password_hash, full_name, role)
58    VALUES ('txn.demo2@mail.example', :demo_hash, 'Txn Demo Two', 'candidate');
59
60    INSERT INTO candidates (user_id, headline)
61    SELECT user_id, 'Should never be saved' FROM users WHERE email = 'txn.demo2@mail.example';
62
63    INSERT INTO applications (job_id, candidate_id)
64    SELECT j.job_id, u.user_id FROM jobs j, users u
65    WHERE j.title = 'Data Analyst' AND u.email = 'txn.demo2@mail.example';
66
67    \echo '-- Inside the transaction (uncommitted), the new rows are visible:'
68    SELECT u.email, j.applicant_count
69    FROM users u, jobs j WHERE u.email = 'txn.demo2@mail.example' AND j.title = 'Data Analyst';
70
71    \echo '-- Controlled failure: apply to the same job again -> UNIQUE (job_id, candidate_id) violation'
72    INSERT INTO applications (job_id, candidate_id)
73    SELECT j.job_id, u.user_id FROM jobs j, users u
74    WHERE j.title = 'Data Analyst' AND u.email = 'txn.demo2@mail.example';
75
76    \echo '-- Once a statement fails, PostgreSQL refuses further work in this transaction:'
77    SELECT 1;
78  ROLLBACK;
79
80  \echo '-- After ROLLBACK: nothing from Part B exists and applicant_count is unchanged:'
81  SELECT count(*) AS demo2_users FROM users WHERE email = 'txn.demo2@mail.example';
82  SELECT count(*) AS demo2_candidates FROM candidates c JOIN users u USING (user_id) WHERE u.email =
    'txn.demo2@mail.example';
83  SELECT title, applicant_count FROM jobs WHERE title = 'Data Analyst';
```

OUTPUT psql, seeded TalentMatch database

```
BEGIN
INSERT 0 1
INSERT 0 1
INSERT 0 1
-- Inside the transaction (uncommitted), the new rows are visible:
         email          | applicant_count
------------------------+-----------------
 txn.demo2@mail.example |               4
(1 row)

-- Controlled failure: apply to the same job again -> UNIQUE (job_id, candidate_id) violation
ERROR:  duplicate key value violates unique constraint "applications_job_id_candidate_id_key"
DETAIL:  Key (job_id, candidate_id)=(2, 36) already exists.
-- Once a statement fails, PostgreSQL refuses further work in this transaction:
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ROLLBACK
-- After ROLLBACK: nothing from Part B exists and applicant_count is unchanged:
 demo2_users
-------------
           0
(1 row)

 demo2_candidates
------------------
                0
(1 row)

    title     | applicant_count
--------------+-----------------
 Data Analyst |               3
(1 row)
```

06 NoSQL / Vector Database Commands

TalentMatch uses **Option 2: SQL + Vector Database**. The vector store is pgvector inside the same PostgreSQL database, so every vector sits under a real foreign key of the resume or job it describes, and ranking is one SQL query that combines vector distance with relational filters (primary resume, active user, applicants only, skills).

6.1 Vector design

QUESTION (HANDOUT §8)	TALENTMATCH
What is embedded	Resume: `resumes.extracted_text` (text extracted from the uploaded PDF or TXT). Job: title + description + the names of its *required* skills.
Embedding model	`sentence-transformers/all-MiniLM-L6-v2`, run inside Node.js with `@huggingface/transformers`. 384 dimensions. Text is split into chunks of at most 254 tokens with 32 tokens of overlap; chunk vectors are averaged (weighted by tokens) and L2-normalised.
How vectors are stored	`resume_embeddings` and `job_embeddings`: one `vector(384)` per resume or job, primary key = foreign key with ON DELETE CASCADE, plus model name and a SHA-256 of the embedded text.
Similarity metric	Cosine similarity = `1 - (a <=> b)`, where `<=>` is pgvector's cosine distance.
How results are retrieved	`rank_candidates_for_job()`: `ORDER BY embedding <=> job_vector LIMIT k` (the shape the HNSW index serves), joined with skills and applications. `compute_match_score()` stores the score on each new application.

6.2 Vector DDL

CREATE EXTENSION — enable pgvector

Migration 001 runs before any vector column exists (the Docker init script does the same on first start).

db/migrations/001_enable_extensions.sql — lines 4

```
4  CREATE EXTENSION IF NOT EXISTS vector;
```

CREATE TABLE — resume_embeddings — vector(384) with integrity rules

`job_embeddings` (lines 27–41) has the same structure, keyed by `job_id`. The CHECK on `vector_norm` only accepts unit-length vectors, so cosine maths stays valid even if application code had a bug.

db/migrations/007_vector_embeddings.sql — lines 9–25

```
9  CREATE TABLE IF NOT EXISTS resume_embeddings (
10     resume_id    BIGINT       NOT NULL,
11     embedding    vector(384)  NOT NULL,
12     model_name   VARCHAR(100) NOT NULL DEFAULT 'sentence-transformers/all-MiniLM-L6-v2',
13     chunk_count  SMALLINT     NOT NULL,
14     -- SHA-256 of the text that was embedded; lets us detect stale embeddings.
15     source_hash  CHAR(64)     NOT NULL,
16     created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
17
18     CONSTRAINT resume_embeddings_pkey PRIMARY KEY (resume_id),
19     CONSTRAINT resume_embeddings_chunk_count_check CHECK (chunk_count >= 1),
20     CONSTRAINT resume_embeddings_source_hash_check CHECK (source_hash ~ '^[0-9a-f]{64}$'),
21     -- Stored vectors must be normalised (length ≈ 1).
22     CONSTRAINT resume_embeddings_normalized_check CHECK (abs(vector_norm(embedding) - 1) < 0.001),
23     CONSTRAINT resume_embeddings_resume_fkey FOREIGN KEY (resume_id)
24         REFERENCES resumes (resume_id) ON DELETE CASCADE
25 );
```

`CREATE INDEX … USING HNSW` approximate nearest-neighbour indexes

`vector_cosine_ops` makes the index serve `ORDER BY embedding <=> $1` queries.

db/migrations/007_vector_embeddings.sql — lines 43–49

```
43  -- HNSW = approximate nearest-neighbour graph index. vector_cosine_ops makes
44  -- it serve ORDER BY embedding <=> $1 (cosine distance) queries.
45  CREATE INDEX IF NOT EXISTS resume_embeddings_embedding_hnsw_idx
46      ON resume_embeddings USING hnsw (embedding vector_cosine_ops);
47
48  CREATE INDEX IF NOT EXISTS job_embeddings_embedding_hnsw_idx
49      ON job_embeddings USING hnsw (embedding vector_cosine_ops);
```

checking the catalog

```
SELECT indexname, substring(indexdef FROM 'USING .*') AS method
  FROM pg_indexes WHERE indexdef LIKE '%hnsw%' ORDER BY 1;
```

OUTPUT — psql, seeded TalentMatch database

```
              indexname               |                 method
--------------------------------------+-----------------------------------------
 job_embeddings_embedding_hnsw_idx    | USING hnsw (embedding vector_cosine_ops)
 resume_embeddings_embedding_hnsw_idx | USING hnsw (embedding vector_cosine_ops)
(2 rows)
```

`CONSTRAINTS IN ACTION` the database refuses bad vectors

A vector that is not unit length, and one with 383 dimensions instead of 384.

invalid vectors

```
-- a vector that is not unit length (all 384 values = 1, length = 19.6)
INSERT INTO resume_embeddings (resume_id, embedding, chunk_count, source_hash)
VALUES (1, array_fill(1, ARRAY[384])::vector, 1, repeat('a', 64));
-- a vector with the wrong number of dimensions
INSERT INTO resume_embeddings (resume_id, embedding, chunk_count, source_hash)
VALUES (1, array_fill(0.05, ARRAY[383])::vector, 1, repeat('a', 64));
```

OUTPUT — psql, seeded TalentMatch database

```
ERROR:  new row for relation "resume_embeddings" violates check constraint "resume_embeddings_normalized_check"
ERROR:  expected 384 dimensions, not 383
```

6.3 Writing vectors

`INSERT … ON CONFLICT` **upsert from the application**

Re-embedding the same resume replaces its vector instead of failing. `$2::vector` casts the text form `[0.01,-0.03,…]` to pgvector's type.

src/lib/vector-store.js — lines 33–43

```
// Insert or replace. ON CONFLICT on the primary key makes re-embedding the
// same resume/job an UPDATE instead of an error.
async function saveResumeEmbedding(resumeId, e, query) {
  await query(
    `INSERT INTO resume_embeddings (resume_id, embedding, model_name, chunk_count, source_hash)
     VALUES ($1, $2::vector, $3, $4, $5)
     ON CONFLICT (resume_id) DO UPDATE
        SET embedding = EXCLUDED.embedding, model_name = EXCLUDED.model_name,
            chunk_count = EXCLUDED.chunk_count, source_hash = EXCLUDED.source_hash, created_at = now()`,
    [resumeId, toPgVector(e.vector), e.modelName, e.chunkCount, e.sourceHash]);
}
```

src/lib/embeddings.js — lines 129–135

```
// pgvector accepts the text form '[0.1,0.2,...]' and casts it with ::vector.
export function toPgVector(vector) {
  if (vector.length !== EMBEDDING_DIMENSIONS || !vector.every(Number.isFinite)) {
    throw new Error(`Expected ${EMBEDDING_DIMENSIONS} finite numbers`);
  }
  return `[${vector.join(",")}]`;
}
```

6.4 Similarity search

`FUNCTION` **compute_match_score() — cosine similarity of one resume and one job**

Called inside the apply transaction: `INSERT … VALUES (…, compute_match_score($3, $1))`. Returns NULL when either side has no embedding yet; scores are never invented.

db/migrations/016_compute_match_score.sql — lines 18–28

```
CREATE OR REPLACE FUNCTION compute_match_score(p_resume_id BIGINT, p_job_id BIGINT)
RETURNS NUMERIC
LANGUAGE sql
STABLE
AS $$
    SELECT round((1 - (re.embedding <=> je.embedding))::numeric, 4)
      FROM resume_embeddings re
      CROSS JOIN job_embeddings je
     WHERE re.resume_id = p_resume_id
       AND je.job_id    = p_job_id;
$$;
```

`FUNCTION` **rank_candidates_for_job() — k nearest primary resumes**

Reads the job vector once into a variable, so the main query compares every resume against a constant (the form an index can use), then attaches the candidate's name, experience and required-skill matches.

db/migrations/015_rank_candidates_for_job.sql — lines 21–43, 54–60, 65–101

```
CREATE OR REPLACE FUNCTION rank_candidates_for_job(
    p_job_id          BIGINT,
    p_limit           INTEGER DEFAULT 10,
    p_applicants_only BOOLEAN DEFAULT FALSE
)
RETURNS TABLE (
    candidate_id      BIGINT,
    full_name         VARCHAR,
    years_experience  NUMERIC,
    resume_id         BIGINT,
```

```
    similarity        NUMERIC,
    matched_skills    INTEGER,
    required_skills   INTEGER,
    has_applied       BOOLEAN
)
LANGUAGE plpgsql
STABLE
AS $$
#variable_conflict use_column
DECLARE
    v_job_vector vector(384);
    v_required   INTEGER;
BEGIN
⋮
    -- 2. The job's vector. Read once into a variable so the main query
    --    compares every resume against a constant (lets the HNSW index be used).
    SELECT je.embedding INTO v_job_vector FROM job_embeddings je WHERE je.job_id = p_job_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Job % has no embedding yet', p_job_id
            USING ERRCODE = 'no_data_found', HINT = 'Run npm run db:embed';
    END IF;
⋮
    -- 3. Nearest primary resumes first, then attach profile + skill counts.
    RETURN QUERY
    WITH nearest AS (
        SELECT r.candidate_id, r.resume_id,
               re.embedding <=> v_job_vector AS distance
          FROM resume_embeddings re
          JOIN resumes r ON r.resume_id = re.resume_id
          JOIN users   u ON u.user_id   = r.candidate_id
         WHERE r.is_primary
           AND u.is_active
           AND (NOT p_applicants_only OR EXISTS (
                    SELECT 1 FROM applications a
                     WHERE a.job_id = p_job_id AND a.candidate_id = r.candidate_id))
         ORDER BY re.embedding <=> v_job_vector
         LIMIT p_limit
    )
    SELECT n.candidate_id,
           u.full_name,
           c.years_experience,
           n.resume_id,
           -- 4 decimals: same precision as applications.match_score NUMERIC(5,4).
           round((1 - n.distance)::numeric, 4),
           (SELECT count(*)::int
              FROM job_skills js
              JOIN candidate_skills cs
                ON cs.skill_id = js.skill_id AND cs.candidate_id = n.candidate_id
             WHERE js.job_id = p_job_id
               AND js.is_required
               AND cs.proficiency >= js.min_proficiency),
           v_required,
           EXISTS (SELECT 1 FROM applications a
                    WHERE a.job_id = p_job_id AND a.candidate_id = n.candidate_id)
      FROM nearest n
      JOIN candidates c ON c.user_id = n.candidate_id
      JOIN users      u ON u.user_id = n.candidate_id
     -- Order by the unrounded distance so near-ties keep their true order.
     ORDER BY n.distance, n.candidate_id;
```

SELECT ranking the Machine Learning Engineer job (Q20)

db/queries/important_queries.sql — lines 300–302

```
300  SELECT full_name, years_experience, similarity, matched_skills, required_skills, has_applied
301  FROM rank_candidates_for_job(
302           (SELECT job_id FROM jobs WHERE title = 'Machine Learning Engineer'), 5, FALSE);
```

RECORDED RESULT — docs/vector-design.md §7, test case 5-19 (PostgreSQL 17 + pgvector 0.8.7)

```
full_name            similarity  matched/required  applied
 1  Aditi Sharma          0.7819      3/3               yes
 2  Rohan Verma           0.6658      1/3               yes
 3  Divya Krishnan        0.5032      3/3               yes
 4  Ishita Banerjee       0.4378      0/3
 5  Aman Malhotra         0.4241      0/3               yes
 …
12  Sneha Kulkarni        0.2438      0/3
```

The same function for the **Frontend Developer (React / Next.js)** job reverses the order: Sneha Kulkarni is first (0.7552, 3/3) and Aditi Sharma eighth (0.2619). Rohan Verma is second for the ML job with only 1 of 3 required skills at the required level: his resume describes deep learning and forecasting work that the vector recognises but a strict skill filter would miss. This is why the system shows both signals.

SELECT · K-NN reverse direction: best open jobs for one candidate (Q22)

The raw k-nearest-neighbour shape `ORDER BY embedding <=> vector LIMIT k` written by hand, combined with a relational filter on open jobs.

db/queries/important_queries.sql — lines 319–330

```
319  SELECT j.title, c.name AS company,
320         round((1 - (je.embedding <=> q.v))::numeric, 4) AS similarity
321  FROM job_embeddings je
322  JOIN jobs j      ON j.job_id = je.job_id AND j.status = 'open'
323  JOIN companies c ON c.company_id = j.company_id
324  CROSS JOIN (SELECT re.embedding AS v
325                FROM resume_embeddings re
326                JOIN resumes r ON r.resume_id = re.resume_id AND r.is_primary
327                JOIN users u   ON u.user_id = r.candidate_id
328               WHERE u.email = 'aditi.sharma@mail.example') AS q
329  ORDER BY je.embedding <=> q.v
330  LIMIT 3;
```

EXPLAIN ANALYZE is the HNSW index used?

With 12 resumes the planner correctly prefers reading all 12 vectors (sequential scan, 0.13 ms). Forcing the index shows it works and returns the same top 5 (1, 2, 11, 7, 12). As the table grows, the planner switches to the index with no query change.

RECORDED PLAN — docs/vector-design.md §8

```
SET LOCAL enable_seqscan = off;   -- plan C

Limit (actual time=1.126..1.137 rows=5)
  -> Index Scan using resume_embeddings_embedding_hnsw_idx on resume_embeddings
       Order By: (embedding <=> (InitPlan 1).col1)
Execution Time: 1.169 ms
```

07 Next.js – Project Tree Structure

The project follows the Next.js **App Router** layout. Inside `src/app`, a folder with a `page.js` is a page and a folder with a `route.js` is an API endpoint (Route Handler), so the folder path is the URL. Browser code lives in `src/components`; everything that talks to the database lives in `src/lib` and is imported only by server code.

7.1 Repository

```
TalentMatch/
├── db/
│   ├── init/001_enable_pgvector.sql     enables pgvector on first start
│   ├── migrations/                      24 ordered migrations
│   │   ├── 001_enable_extensions.sql
│   │   ├── 002_users_companies_profiles.sql
│   │   ├── …                            003 – 023
│   │   └── 024_display_dates_in_rule_messages.sql
│   ├── seed/seed.sql                    fictional demo data
│   ├── queries/important_queries.sql    Q1 – Q41
│   ├── demos/transaction_demo.sql       COMMIT, ROLLBACK, SAVEPOINT
│   ├── tests/
│   │   ├── stage1_constraint_tests.sql
│   │   └── stage2_advanced_tests.sql
│   └── reset.sql                        DROP views, tables, functions
├── docs/                                report, ER diagram, schema, tests, screenshots
├── new_changes/                         handout answers, statistics, diagrams
├── public/                              static icons
├── scripts/
│   ├── lib/                             db-target.js, uploads.js
│   ├── migrate.js                       applies db/migrations in order
│   ├── seed.js
│   ├── embed-seed.js                    embeds seeded resumes and jobs
│   └── reset.js
├── src/                                 the Next.js application (below)
├── tests/
│   ├── unit/                            8 test files
│   ├── integration/                     7 test files + helpers.js
│   └── fixtures/                        sample-resume.pdf / .html
├── .env.example                         template; .env.local is gitignored
├── .gitignore
├── docker-compose.yml                   PostgreSQL 17 + pgvector container
├── jsconfig.json · next.config.mjs
├── package.json · package-lock.json
└── README.md                            setup and run instructions
```

7.2 Application source: src/

Generated from the repository (109 files). A folder that holds a single file is shown on one line, e.g. `login/page.js`.

```
src/
├── app/  App Router: pages + API
│   ├── admin/  admin pages
│   │   ├── companies/page.js
│   │   ├── recruiters/page.js
│   │   └── page.js
│   ├── api/  Route Handlers (backend API)
│   │   ├── admin/  admin only
│   │   │   ├── companies/
│   │   │   │   ├── [companyId]/route.js
│   │   │   │   └── route.js
│   │   │   ├── dashboard/route.js
│   │   │   └── recruiters/
│   │   │       ├── [recruiterId]/route.js
│   │   │       └── route.js
│   │   ├── auth/  login, logout, register, me
│   │   │   ├── login/route.js
│   │   │   ├── logout/route.js
│   │   │   ├── me/route.js
│   │   │   └── register/route.js
│   │   ├── candidate/  candidate only
│   │   │   ├── applications/route.js  apply (transaction)
│   │   │   ├── interviews/route.js
│   │   │   ├── jobs/
│   │   │   │   ├── [jobId]/route.js
│   │   │   │   └── route.js
│   │   │   ├── offers/
│   │   │   │   ├── [offerId]/respond/route.js
│   │   │   │   └── route.js
│   │   │   ├── profile/route.js
│   │   │   ├── resumes/
│   │   │   │   ├── [resumeId]/route.js
│   │   │   │   └── route.js
│   │   │   └── skills/
│   │   │       ├── [skillId]/route.js
│   │   │       └── route.js
│   │   ├── health/route.js
│   │   ├── notifications/
│   │   │   ├── [notificationId]/route.js
│   │   │   ├── read-all/route.js
│   │   │   └── route.js
│   │   ├── recruiter/  recruiter only
│   │   │   ├── applications/[applicationId]/
│   │   │   │   ├── interviews/route.js
│   │   │   │   ├── offer/route.js
│   │   │   │   └── route.js
│   │   │   ├── interviews/
│   │   │   │   ├── [interviewId]/route.js
│   │   │   │   └── route.js
│   │   │   ├── jobs/
│   │   │   │   ├── [jobId]/
│   │   │   │   │   ├── applications/route.js
│   │   │   │   │   ├── matches/route.js  semantic ranking
│   │   │   │   │   └── route.js
│   │   │   │   └── route.js
│   │   │   └── offers/
│   │   │       ├── [offerId]/withdraw/route.js
│   │   │       └── route.js
│   │   └── skills/route.js
│   ├── candidate/  candidate pages
│   │   ├── applications/page.js
│   │   ├── interviews/page.js
│   │   ├── jobs/
│   │   │   ├── [jobId]/page.js
│   │   │   └── page.js
│   │   ├── offers/page.js
│   │   └── page.js
│   ├── login/page.js
│   ├── notifications/page.js
│   ├── recruiter/  recruiter pages
│   │   ├── interviews/page.js
│   │   ├── jobs/
│   │   │   ├── [jobId]/
│   │   │   │   ├── edit/page.js
│   │   │   │   └── page.js
│   │   │   └── new/page.js
│   │   ├── offers/page.js
│   │   └── page.js
│   ├── register/page.js
│   ├── favicon.ico
│   ├── globals.css
│   ├── layout.js  root layout
│   └── page.js  landing page
├── components/  React client components
│   ├── admin/
│   │   ├── AdminDashboard.js
│   │   ├── CompanyManager.js
│   │   └── RecruiterManager.js
│   ├── auth/
│   │   ├── AuthFormParts.js
│   │   ├── LoginForm.js
│   │   └── RegisterForm.js
│   ├── candidate/
│   │   ├── CandidateDashboard.js
│   │   ├── CandidateInterviews.js
│   │   ├── CandidateJobDetail.js
│   │   ├── CandidateOffers.js
│   │   ├── JobBrowser.js
│   │   └── MyApplications.js
│   ├── nav/TopNav.js
│   ├── recruiter/
│   │   ├── JobForm.js
│   │   ├── PipelineForms.js
│   │   ├── RecruiterInterviews.js
│   │   ├── RecruiterJobDetail.js
│   │   ├── RecruiterJobs.js
│   │   └── RecruiterOffers.js
│   ├── charts.js  dashboard charts
│   ├── NotificationsInbox.js
│   └── ui.js
└── lib/  server-side logic
    ├── admin/
    │   ├── companies.js
    │   ├── dashboard.js  analytics SQL
    │   ├── recruiters.js
    │   └── validation.js
    ├── api/
    │   ├── api-error.js
    │   ├── db-rules.js
    │   ├── errors.js
    │   └── request.js
    ├── auth/
    │   ├── cookie.js
    │   ├── guards.js  withAuth: 401 / 403
    │   ├── jwt.js  HS256 session token
    │   ├── login-rate-limit.js  5 tries / 15 min
    │   ├── password.js  bcrypt (cost 12)
    │   └── session.js
    ├── candidate/
    │   ├── resumes.js
    │   ├── skill-gap.js  EXCEPT query
    │   └── validation.js
    ├── recruiter/
    │   ├── jobs.js
    │   └── validation.js
    ├── validation/
    │   ├── auth.js
    │   └── pipeline.js
    ├── applications.js  apply-to-job transaction
    ├── db.js  pg Pool from DATABASE_URL
    ├── embeddings.js  MiniLM, chunking, pooling
    ├── format-date.js
    ├── interviews.js  interview transactions
    ├── offers.js  offer transactions
    ├── roles.js
    └── vector-store.js  pgvector upserts
```

08 Sample Screenshots of Application Code and Demo

8.1 shows the code that connects the interface to the database; 8.2 shows the running application with the seeded demo data. All 34 screenshots are in `docs/screenshots/final/` (numbers below match the file names); tall pages are shown from the top.

8.1 Application code

DATABASE CONNECTION **one pool, credentials from the environment**

`DATABASE_URL` comes from `.env.local`, which is gitignored; only `.env.example` with placeholders is committed.

src/lib/db.js — lines 1–24

```
import { Pool } from "pg";

// One shared connection pool for the whole server.
// In development, Next.js hot-reloads modules, so the pool is cached on
// globalThis to avoid opening a new set of connections on every reload.
const globalForDb = globalThis;

export function getPool() {
  if (!globalForDb.talentmatchPool) {
    const connectionString = process.env.DATABASE_URL;
    if (!connectionString) {
      throw new Error(
        "DATABASE_URL is not set. Copy .env.example to .env.local and fill it in."
      );
    }
    globalForDb.talentmatchPool = new Pool({ connectionString, max: 10 });
  }
  return globalForDb.talentmatchPool;
}

// Run a parameterized query: query("SELECT * FROM jobs WHERE id = $1", [id])
export function query(text, params) {
  return getPool().query(text, params);
}
```

PASSWORD HASHING **bcrypt with a per-password salt**

Unknown e-mail and wrong password take the same time, so login cannot be used to find out which e-mails exist.

src/lib/auth/password.js — lines 18–34

```
export async function hashPassword(plain) {
  if (typeof plain !== "string" || plain.length === 0) {
    throw new TypeError("Password must be a non-empty string");
  }
  if (Buffer.byteLength(plain, "utf8") > MAX_PASSWORD_BYTES) {
    throw new RangeError(`Password must be at most ${MAX_PASSWORD_BYTES} bytes`);
  }
  return bcrypt.hash(plain, BCRYPT_COST); // salt is generated and embedded automatically
}

// Returns true/false; never throws for a wrong password. Pass hash = null when
// the user was not found to keep timing uniform.
export async function verifyPassword(plain, hash) {
  if (typeof plain !== "string") return false;
  const ok = await bcrypt.compare(plain, hash ?? DUMMY_HASH);
  return hash ? ok : false;
}
```

AUTHORISATION **withAuth — every protected Route Handler is wrapped**

401 without a valid session, 403 for the wrong role, checked on the server before the handler runs.

src/lib/auth/guards.js lines 54–68

```
54  export function withAuth(options, handler) {
55    const roles = options?.roles;
56    return async function authenticatedRoute(request, context) {
57      try {
58        await connection();
59        const user = await requireUser(request);
60        if (roles) assertRole(user, roles);
61        const params = context?.params ? await context.params : {};
62        return await handler(request, { user, params });
63      } catch (err) {
64        unstable_rethrow(err);
65        return handleApiError(err);
66      }
67    };
68  }
```

ROUTE HANDLER **POST /api/candidate/applications**

Role check, same-origin check, zod validation, then the transaction below. The candidate id comes from the session, never from the request body.

src/app/api/candidate/applications/route.js lines 33–40

```
33  // POST /api/candidate/applications { jobId, resumeId, coverLetter? }
34  // Runs the apply transaction in src/lib/applications.js.
35  export const POST = withAuth({ roles: [ROLES.CANDIDATE] }, async (request, { user }) => {
36    assertSameOrigin(request);
37    const data = await readJsonBody(request, applicationCreateSchema);
38    const application = await applyToJob({ candidateId: user.id, ...data });
39    return NextResponse.json({ application }, { status: 201, headers: { "Cache-Control": "no-store" } });
40  });
```

TRANSACTION **applyToJob — BEGIN ... COMMIT, ROLLBACK on any error**

Locks the job (FOR SHARE), checks the resume belongs to the candidate, inserts the application with a real cosine score, and notifies the recruiter. Three triggers fire inside the same transaction.

src/lib/applications.js lines 36–90

```
36  export async function applyToJob(
37    { candidateId, jobId, resumeId, coverLetter = null },
38    { pool = getPool(), notification = defaultApplicationNotification } = {},
39  ) {
40    const client = await pool.connect();
41    try {
42      await client.query("BEGIN");
43
44      const job = await client.query(
45        "SELECT job_id, title, status, company_id, recruiter_id FROM jobs WHERE job_id = $1 FOR SHARE", [jobId]);
46      // Draft jobs are not published, so to a candidate they do not exist.
47      if (!job.rowCount || job.rows[0].status === "draft") throw notFound("Job not found");
48      if (job.rows[0].status !== "open") {
49        throw new ApiError(409, "JOB_NOT_OPEN", "This job is closed and no longer accepts applications");
50      }
51      const { title: jobTitle, company_id: companyId, recruiter_id: recruiterId } = job.rows[0];
52
53      const resume = await client.query(
54        "SELECT resume_id FROM resumes WHERE resume_id = $1 AND candidate_id = $2 FOR SHARE", [resumeId,
    candidateId]);
55      if (!resume.rowCount) throw notFound("Resume not found");
56
57      // Real cosine similarity computed in the database; NULL if the resume or
58      // the job has no embedding yet. applicant_count is NOT touched here: the
59      // AFTER INSERT trigger maintains it.
60      const inserted = await client.query(
```

```
      `INSERT INTO applications (job_id, candidate_id, resume_id, cover_letter, match_score)
       VALUES ($1, $2, $3, $4, compute_match_score($3, $1))
       RETURNING application_id::int AS id, job_id::int AS "jobId", resume_id::int AS "resumeId",
                 status, match_score::float8 AS "matchScore", applied_at AS "appliedAt"`,
      [jobId, candidateId, resumeId, coverLetter]);
    const application = inserted.rows[0];

    const candidate = await client.query("SELECT full_name FROM users WHERE user_id = $1", [candidateId]);
    const note = notification({ jobTitle, candidateName: candidate.rows[0].full_name });
    // The recruiter who posted the job; if that recruiter was removed
    // (recruiter_id NULL), every recruiter of the company instead.
    await client.query(
      `INSERT INTO notifications (user_id, type, title, message, related_application_id)
       SELECT r.user_id, 'application_submitted', $3, $4, $5
         FROM recruiters r
        WHERE r.company_id = $1 AND ($2::bigint IS NULL OR r.user_id = $2)`,
      [companyId, recruiterId, note.title, note.message, application.id]);

    await client.query("COMMIT");
    return application;
  } catch (err) {
    await client.query("ROLLBACK").catch(() => {});
    if (err?.code === "23505" && err?.constraint === "applications_job_id_candidate_id_key") {
      throw new ApiError(409, "ALREADY_APPLIED", "You have already applied to this job");
    }
    throw err;
  } finally {
    client.release();
  }
}
```

EMBEDDINGS

embedText — text to one 384-number unit vector

src/lib/embeddings.js — lines 137–161

```
// Embed one document. Returns everything the *_embeddings tables store.
export async function embedText(rawText) {
  const text = normalizeText(rawText);
  if (!text) throw new Error("Cannot embed empty text");
  const extractor = await getExtractor();
  const countTokens = (word) => extractor.tokenizer.encode(word, { add_special_tokens: false }).length;
  const { chunks, truncated } = chunkWords(text.split(" "), countTokens);

  const chunkVectors = [];
  for (let i = 0; i < chunks.length; i += BATCH_SIZE) {
    const batch = chunks.slice(i, i + BATCH_SIZE).map((c) => c.text);
    const output = await extractor(batch, { pooling: "mean", normalize: true });
    const [rows, dims] = output.dims;
    if (dims !== EMBEDDING_DIMENSIONS) throw new Error(`Model returned ${dims} dimensions, expected
${EMBEDDING_DIMENSIONS}`);
    for (let r = 0; r < rows; r++) chunkVectors.push(output.data.subarray(r * dims, (r + 1) * dims));
  }

  return {
    vector: weightedMeanNormalized(chunkVectors, chunks.map((c) => c.tokens)),
    chunkCount: chunks.length,
    truncated,
    modelName: EMBEDDING_MODEL,
    sourceHash: sha256Hex(text),
  };
}
```

SEMANTIC RANKING API

GET /api/recruiter/jobs/:jobId/matches

The recruiter's company and the job's company are both read from the database; the ranking itself comes from `rank_candidates_for_job()`.

src/app/api/recruiter/jobs/[jobId]/matches/route.js — lines 12–32, 38–53

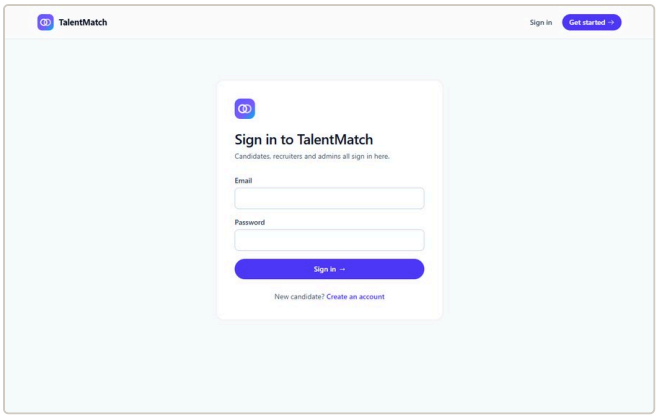
```
export const GET = withAuth({ roles: [ROLES.RECRUITER] }, async (request, { user, params }) => {
  const jobId = idParamSchema.safeParse(params.jobId);
```

```
  if (!jobId.success) throw notFound("Job not found");
  const options = jobMatchesQuerySchema.parse(Object.fromEntries(request.nextUrl.searchParams));

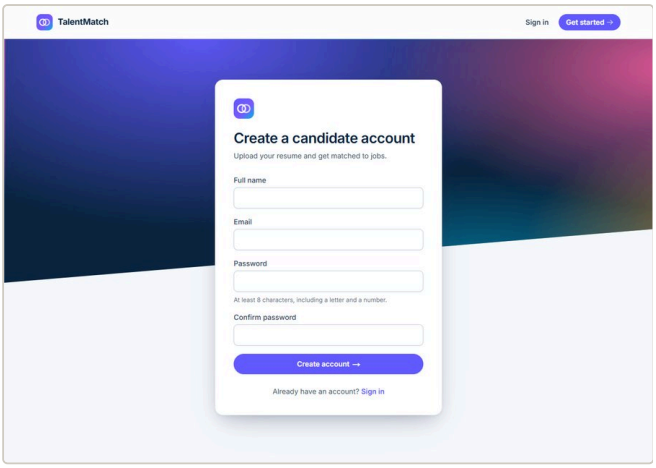
  // Ownership comes from the database: the job's company and the logged-in
  // recruiter's company, never from anything the client sends.
  const { rows } = await query(
    `SELECT j.title, j.company_id, je.model_name, je.created_at AS embedded_at,
            (SELECT r.company_id FROM recruiters r WHERE r.user_id = $2) AS recruiter_company_id
       FROM jobs j
       LEFT JOIN job_embeddings je ON je.job_id = j.job_id
      WHERE j.job_id = $1`, [jobId.data, user.id]);
  if (!rows.length) throw notFound("Job not found");
  const job = rows[0];
  if (job.recruiter_company_id === null || String(job.recruiter_company_id) !== String(job.company_id)) {
    throw forbidden("You can only view matches for your own company's jobs");
  }
  if (job.model_name === null) {
    throw new ApiError(404, "NOT_EMBEDDED", "This job has no embedding yet, so it cannot be matched. Run `npm run db:embed` or save the job again.");
  }
⋮
  const matches = await query(
    `SELECT m.candidate_id::int AS "candidateId", m.full_name AS "fullName",
            m.years_experience::float8 AS "yearsExperience", m.resume_id::int AS "resumeId",
            m.similarity::float8 AS similarity, m.matched_skills AS "matchedSkills",
            m.required_skills AS "requiredSkills", m.has_applied AS "hasApplied",
            COALESCE((SELECT array_agg(s.name ORDER BY s.name)
                        FROM job_skills js
                        JOIN skills s ON s.skill_id = js.skill_id
                        JOIN candidate_skills cs ON cs.skill_id = js.skill_id AND cs.candidate_id = m.candidate_id
                       WHERE js.job_id = $1 AND js.is_required
                         AND cs.proficiency >= js.min_proficiency), '{}') AS "matchedSkillNames",
            a.application_id::int AS "applicationId", a.status AS "applicationStatus"
       FROM rank_candidates_for_job($1, $2, $3) WITH ORDINALITY AS m
       LEFT JOIN applications a ON a.job_id = $1 AND a.candidate_id = m.candidate_id
      ORDER BY m.ordinality`,
    [jobId.data, options.limit, options.applicantsOnly]);
```

8.2 Application demo

AUTHENTICATION

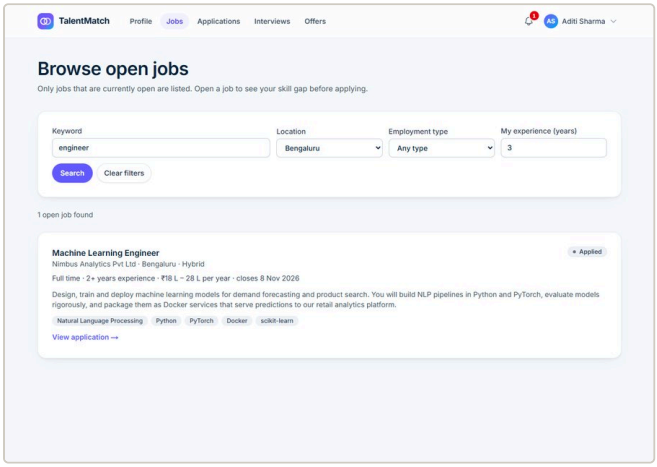


01 Login

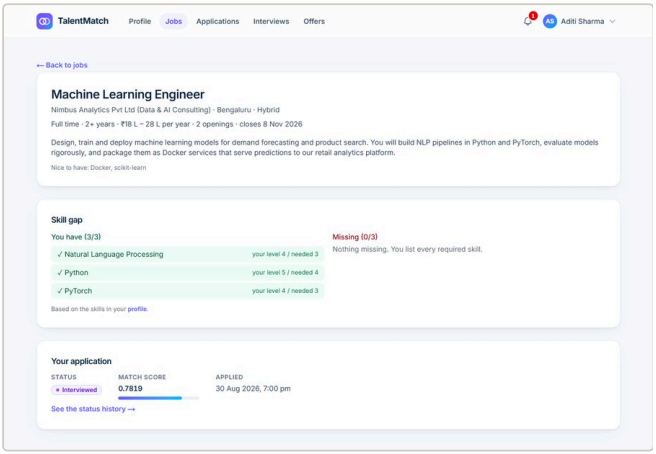


02 Candidate registration

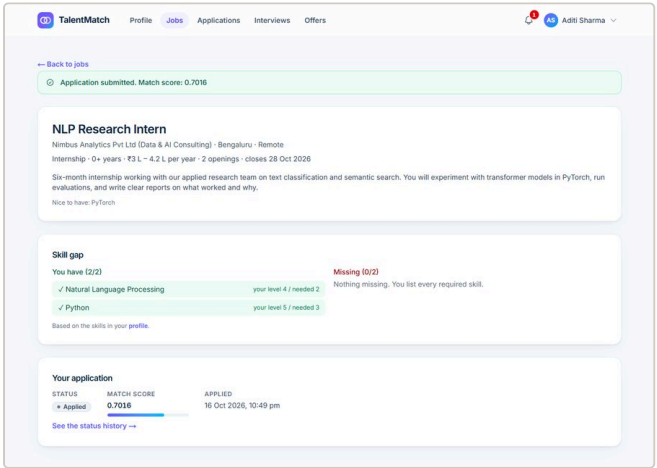
CANDIDATE



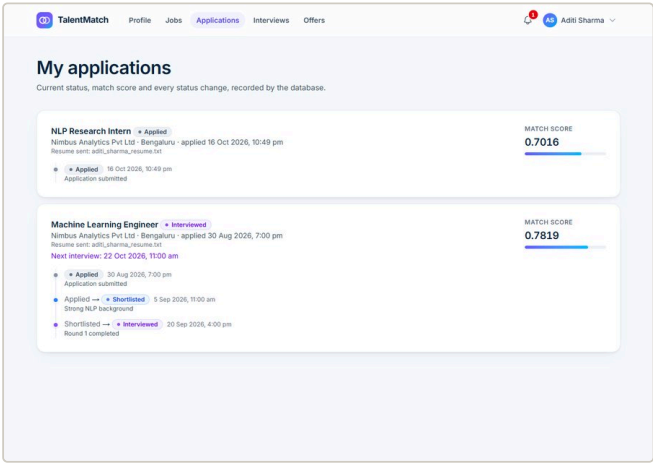
05 Job search with filters



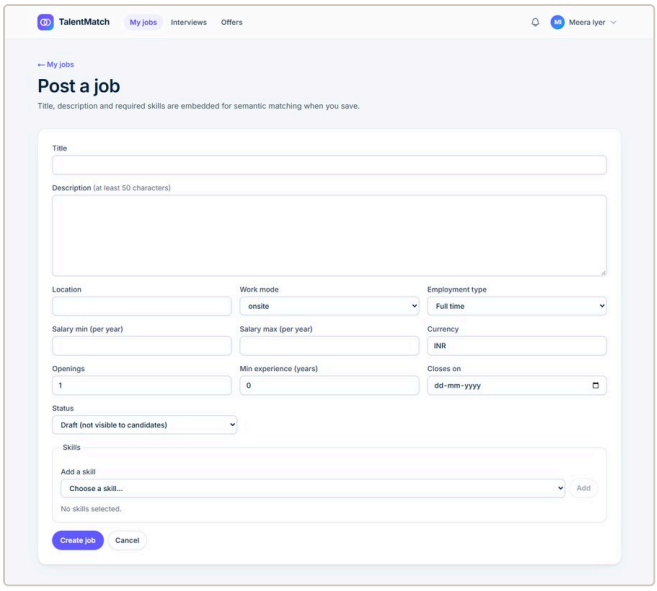
06 Skill gap for a job (EXCEPT query)



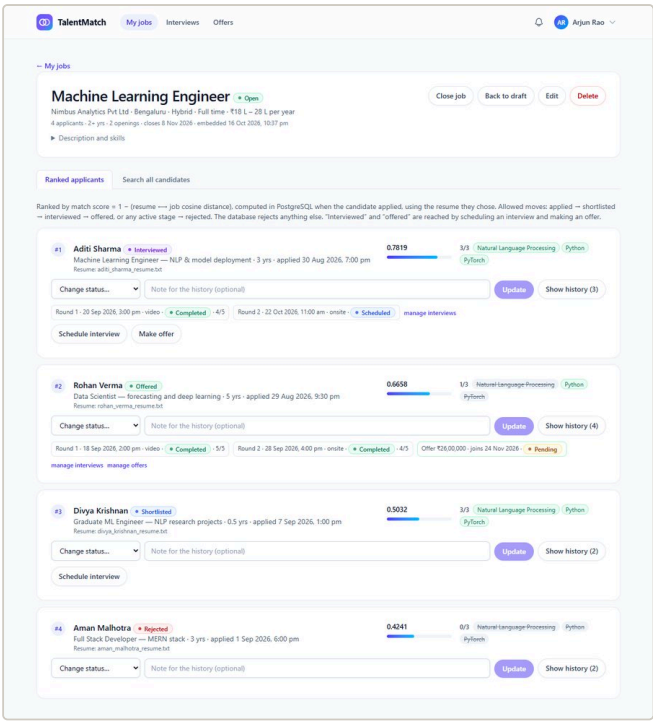
07 Applying: real cosine match score



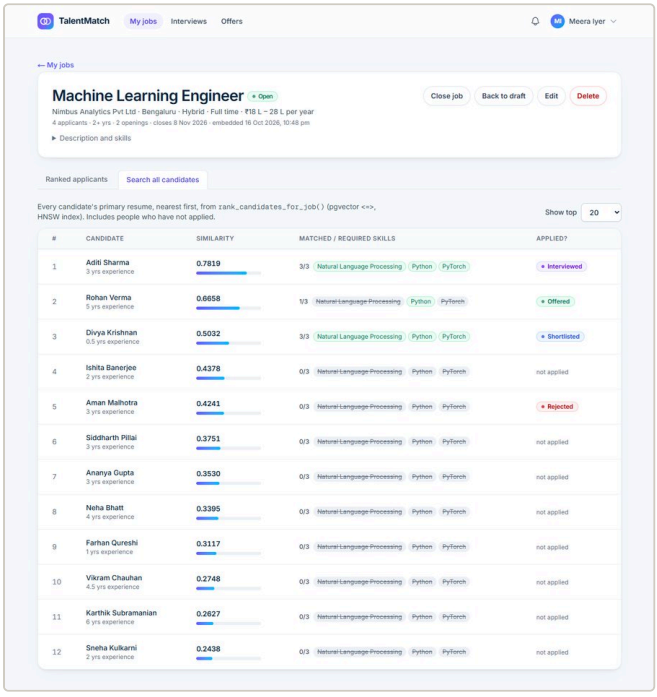
08 My applications with status history



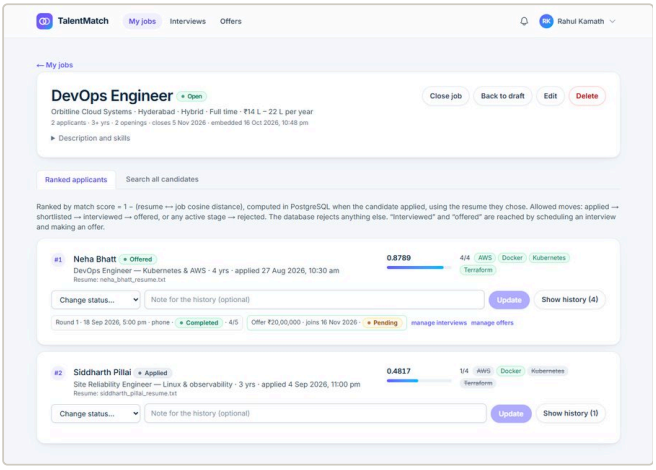
10 Job form with required skills



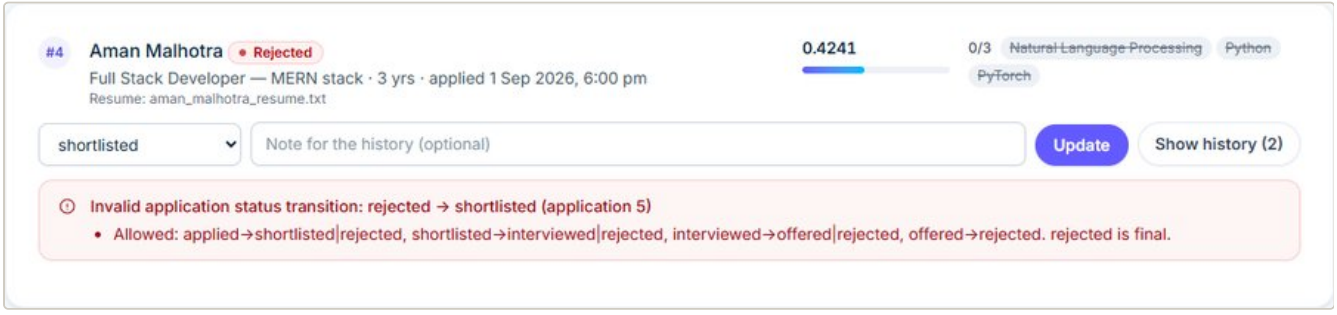
11 Applicants ranked by similarity (pgvector)



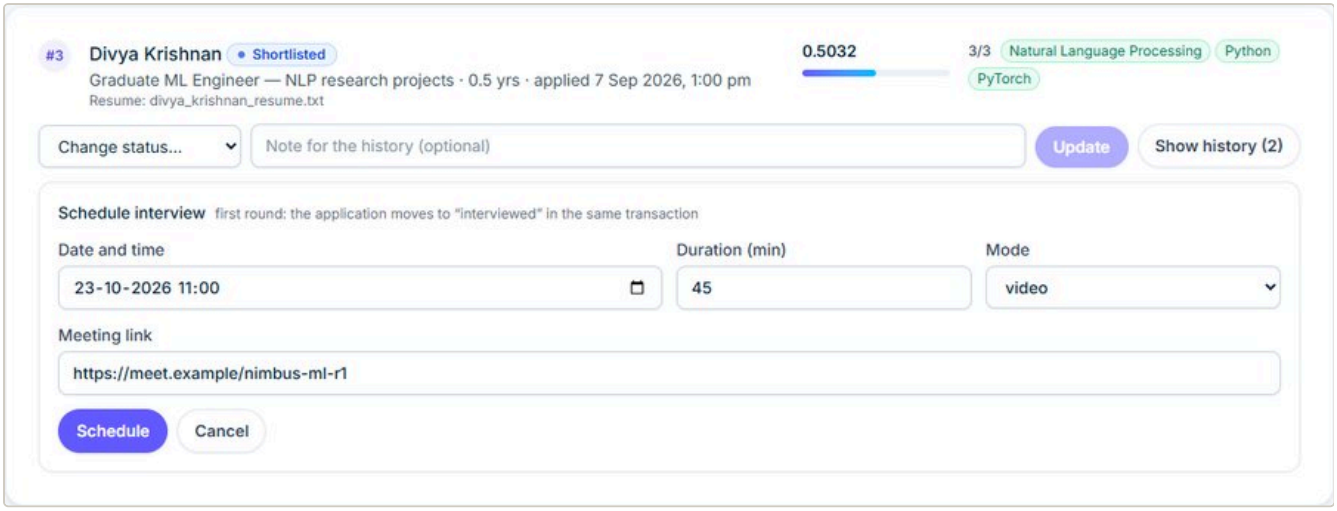
12 Semantic search over all candidates



17 Offer made

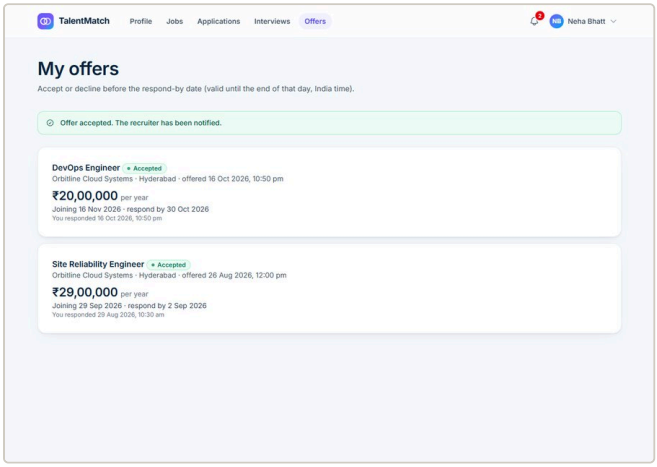


13 Invalid status change refused by the trigger

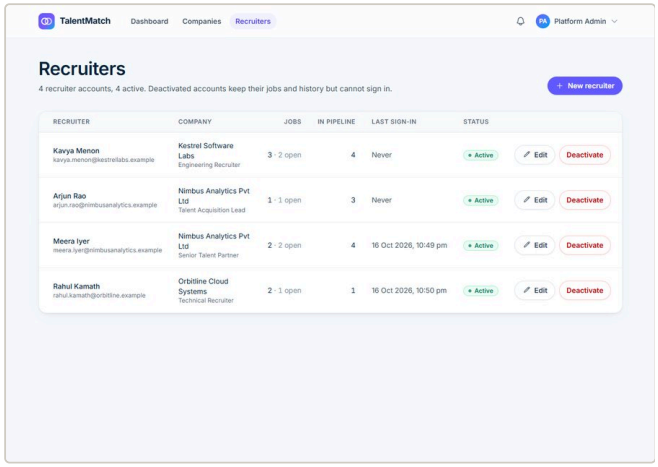


14 Scheduling an interview

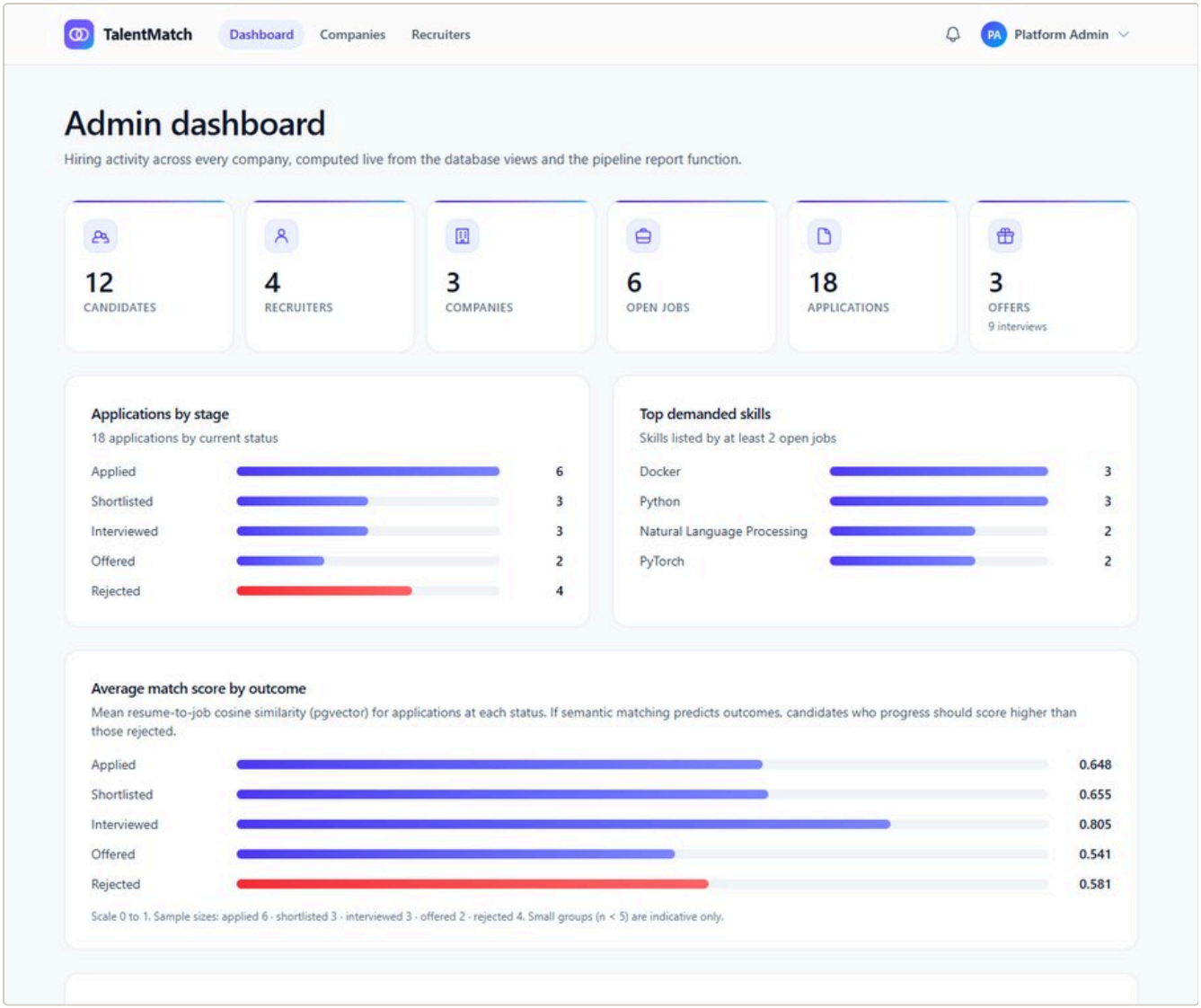
CANDIDATE RESPONSE AND ADMIN



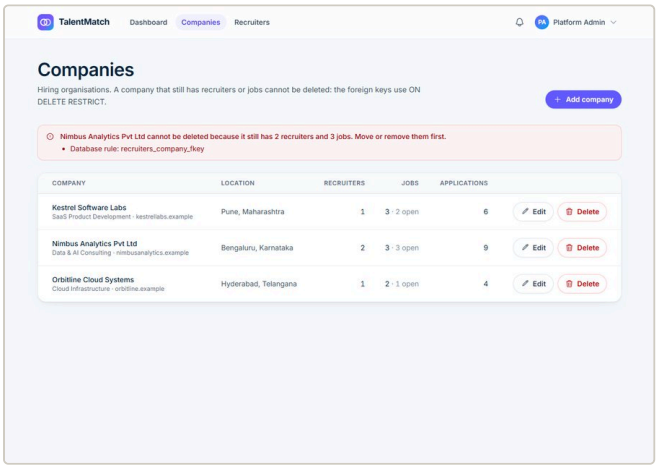
21 Candidate accepts the offer



27 Admin: recruiter accounts

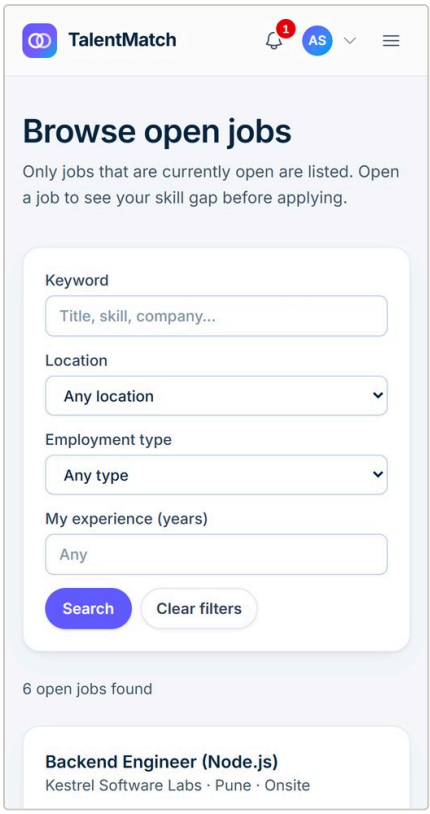


25 Admin dashboard (views + report function)

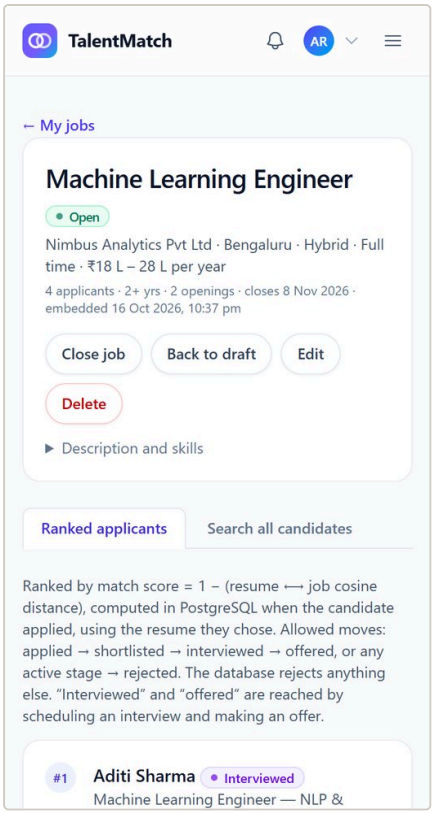


26 Company delete refused (ON DELETE RESTRICT)

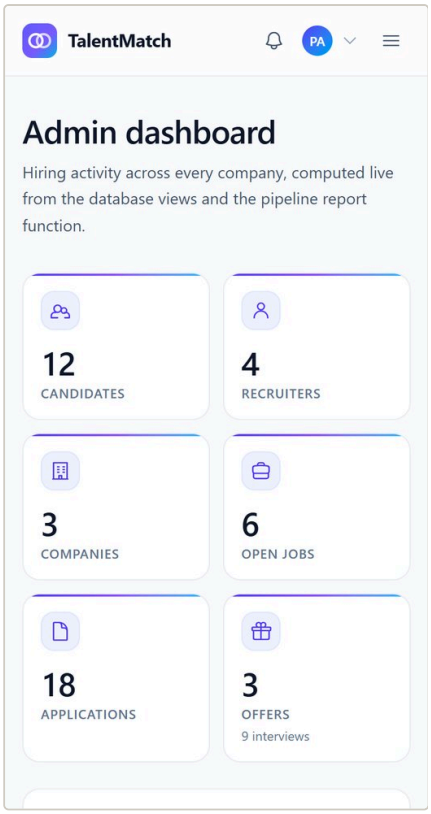
MOBILE LAYOUT



28 Candidate jobs



29 Ranked applicants



30 Admin dashboard

09 Evaluation Rubrics – Completion Status

Status of every requirement in the course handout (DBMS Level 3 Project Requirements, §1–§18), with the place in the repository where it can be checked.

§	HANDOUT REQUIREMENT	STATUS	EVIDENCE
1	Unique title; real-world problem; not simple CRUD	● Met	Semantic resume ranking + hiring workflow enforced in the database
2	Two-member team with defined responsibilities	● Met	Rohith Karning: database, backend/API and semantic matching. Saanvi Maharana: frontend
3	SQL + NoSQL or SQL + Vector database	● Met	Option 2: PostgreSQL 17 + pgvector 0.8.7 (Sections 5 and 6)
4	ER diagram → relational schema: entities, attributes, PK, FK, relationships, constraints; vector data model	● Met	`docs/er-diagram.md`, `new_changes/er-diagram-chen.png`, `docs/relational-schema.md`, `docs/data-dictionary.md`, `docs/vector-design.md`
5	DDL: CREATE, ALTER, DROP, PK/FK, constraints, indexes, views	● Met	Section 5.2 and 5.4; `db/migrations/001–024`, `db/reset.sql`
6	DML: INSERT, UPDATE, DELETE, SELECT; JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	● Met	Section 5.3; 41 queries in `db/queries/important_queries.sql`, all run with exit code 0
7	At least 4 advanced SQL concepts + 1 NoSQL/vector	● Met	Transactions, views (3), stored functions (6), triggers (19), indexing (46, incl. 3 partial) + vector similarity search with HNSW
8	Vector search: data, model, storage, metric, retrieval	● Met	Section 6.1; `docs/vector-design.md`
9	Functional Next.js interface with forms, tables, search, filters, dashboards	● Met	19 app pages for three roles; Section 8.2
10	Backend/API layer; credentials in environment variables	● Met	36 Route Handlers; `DATABASE_URL` and `AUTH_JWT_SECRET` from `.env.local` (gitignored)
11	Authentication; at least 2 roles	● Met	Admin, recruiter, candidate; JWT session cookie; `withAuth` on every protected route
12	Validation at the database level; integrity	● Met	57 CHECK, 22 FK, 8 UNIQUE, partial unique index, guard triggers; zod in the API as a first layer
13	Basic security; no plain-text passwords or credentials	● Met	bcrypt (cost 12) + CHECK on hash format; parameterised SQL; httpOnly SameSite cookie; login rate limit (5 per 15 min); same-origin check on writes
14	Documentation: problem, objectives, scope, FRs, ER, schema, DB justification, data dictionary, implementation, queries, architecture, screenshots, tests, conclusion	● Met	`docs/project-report.md` (+ PDF) covers all 14; requirement-by-requirement answers in `new_changes/handout-requirements.md`
15	Git/GitHub; commits show each member's contribution; README	● Met	Repository with README (setup and run steps). Work split: Rohith Karning on the database, backend/API and semantic matching; Saanvi Maharana on the frontend
16	Demonstration items incl. test cases and contributions	● Met	`docs/viva-demo-script.md`; final run on 16 Oct 2026: 85/85 unit, 147/147 integration, 8/8 constraint tests, 27 PASS advanced SQL tests (`docs/test-cases.md`)

§	HANDOUT REQUIREMENT	STATUS	EVIDENCE
17	Original design and implementation	● Met	Own schema, SQL and code, built stage by stage from the first commit (`f534e15` , 9 Oct 2026)
18	Beyond CRUD; database choice justified by the data	● Met	Workflow rules, ranking and analytics run in the database; justification in `docs/vector-design.md` §9

9.1 Submission checklist

REQUIRED BY THE SUBMISSION NOTICE	WHERE
1 · Project Title	Cover page
2 · Student SRNs and Names	Cover page
3 · Abstract	Section 3
4 · Technology Stack	Section 4
5 · SQL – Database Commands	Section 5 (DDL, DML, views, functions, triggers, transactions)
6 · NoSQL/Vector Database Commands	Section 6 (pgvector DDL, upserts, similarity search, EXPLAIN)
7 · Next.js – Project Tree Structure	Section 7
8 · Screenshots of Application Code and Demo	Section 8
9 · Evaluation Rubrics – Completion Status	Section 9
Deliverable: completed DBMS Project Report	This document
Deliverable: GitHub repository link	`https://github.com/rohithkarning-debug/TalentMatch`

GITHUB REPOSITORY

`https://github.com/rohithkarning-debug/TalentMatch`

Setup: copy `.env.example` to `.env.local`, then `docker compose --env-file .env.local up -d`, `npm install`, `npm run db:migrate`, `npm run db:seed`, `npm run db:embed`, `npm run dev` (full steps in README.md).